

CS7CS3 Advanced Software Engineering Group Project

Project Name: *Response to City Disaster*

Project Diary

Group: 8

*mjaiswal@tcd.ie, mathankr@tcd.ie, courtinm@tcd.ie, fraiolim@tcd.ie,
taurov@tcd.ie, bharambn@tcd.ie, birarit@tcd.ie, kirthivs@tcd.ie, jainpr@tcd.ie,
taraneka@tcd.ie*

1. Project Overview

The Response to City Disaster platform is a cloud-native emergency management system designed to coordinate multi-agency response to urban disaster scenarios. The system provides real-time situational awareness, automated unit dispatching, structured evacuation planning, and citizen communication through two complementary surfaces: a Web Dashboard intended for dispatchers, emergency managers, and administrators, and a Flutter-based Mobile Application for field responders and members of the public.

The system is deployed on Google Cloud Platform (GCP) and uses Google Kubernetes Engine (GKE) as its primary runtime environment. A total of 183 Jira tickets were created and resolved across the project lifecycle, organised under twenty major features spanning infrastructure, backend services, frontend applications, and documentation.

The system integrates real-time data from multiple external sources (transport networks, weather services, power grids, hospital systems) through a dedicated Data ETL service orchestrated by Apache Airflow. All data is persisted and made queryable via a ClickHouse columnar database cluster, enabling high-throughput analytics on incident and observability data.

2. Technical Architecture and Approach

The platform follows a microservices architecture in which each functional domain is encapsulated as an independently deployable service. All services are written in Go, containerised with multi-stage Dockerfiles, and deployed on GKE with Kubernetes Deployments, Horizontal Pod Autoscalers, and ConfigMap/Secret-backed environment management. Inter-service communication is handled through a central Event Bus (publish/subscribe) implemented, with select endpoints exposed via REST APIs for synchronous operations and WebSocket APIs for real-time streaming.

The eight core backend services are: (1) IAM Service, managing OAuth login flows via Keycloak, user profiles, ERT member registration, session refresh, and account status; (2) Core Service, a production-ready microservice for disaster response management encompassing the Incident Registry, Dispatching, Evacuation Planning, and Traffic Management domains, providing full incident lifecycle management, geospatial unit selection and assignment, multi-variable pathfinding and evacuation plan publication, and road closure management; (3) Notification Service, delivering single-target and geo-fenced broadcast alerts with template management; (4) Communication Service, providing a WebSocket/REST hybrid messaging layer with an audit log for real-time inter-service and user-facing messaging; (5) Event Bus, acting as the central asynchronous messaging backbone of the platform, handling topic management, high-throughput batch publishing, and subscription handler registration to decouple producers and consumers across all services; (6) Support Bot, providing an automated conversational interface for citizen support with human operator escalation and flagged conversation management; (7) Data ETL Service, managing configurable data source ingestion with schema normalisation; and (8) Observability Service, aggregating system and city-data metrics for the dashboard.

The Web Dashboard is implemented in React and connects to backend services via the Kubernetes Ingress layer. The Mobile Application is implemented in Flutter and uses the TomTom SDK for map rendering. Authentication across both surfaces is managed by Keycloak, deployed on GKE with

CloudSQL/Postgres persistence. The CI/CD pipeline for each service follows a build-test-push-deploy pattern implemented in GitHub Actions, with GCS used for artefact storage and DAG hosting for Airflow.

The project was executed using an XP methodology with weekly iteration cycles. The Jira board was reviewed and updated every Monday at 10:00 AM, establishing a consistent cadence for iteration planning, task assignment, and progress tracking. Iterations were nominally five working days in duration, though the team operated with a degree of flexibility given the academic context.

3. Labour Division Methodology

The team of ten was divided into five pairs for the duration of the project. A deliberate pair rotation strategy was adopted: pair compositions were kept stable for the first two weeks to establish shared working norms, and were thereafter adjusted on a weekly basis. This rotation served several purposes: it prevented knowledge alignment (ensuring that all team members developed familiarity with both the infrastructure and application layers), it distributed the risk of key-person dependency, and it promoted cross-pollination of coding styles and design approaches.

Epic ownership was assigned at the pair level in Week 1, establishing accountability for each major system component. As pairs rotated, the ownership responsibility was handed over, with the outgoing pair member providing context and documentation to the incoming partner. This handover discipline was supported by the onboarding runbook produced in Week 2 and the Confluence documentation effort in Weeks 11–12.

Within each pair, work was divided according to an informal driver/navigator model: one member would write code while the other reviewed, tested, and documented in real time. Pairs were expected to alternate roles across tasks within an iteration. This approach was particularly effective for the algorithmically complex deliverables in Weeks 9 and 10, where a second pair of eyes materially reduced the number of defects introduced.

4. Time Estimation Approach

Time estimation for the project was structured around a planning assumption of ten hours of work per person per week, as reflected in the team's planning documentation. With five pairs and ten team members, this equates to a planned capacity of one hundred person-hours per week, or a project-wide estimated effort of thirteen hundred person-hours across thirteen weeks (including the break in Week 7, which contributed twelve active weeks of effort).

Estimates were set at the Epic and task level without formal story-point assignment in Jira. Tasks were scoped during the Monday planning session and sized informally based on comparison with previously completed work. To estimate the time per task we overestimated the time of each task in order to have a buffer time that we used in case the work was not completed on time.

The team's practice was to resolve minor bugs inline, within the iteration in which they were discovered, without creating a dedicated ticket. A Jira ticket was only raised when an issue was significant enough to warrant explicit tracking, either because the root cause was unclear, because it cut across multiple services, or because it required coordination between pairs. This means that the 10 tickets formally logged in Weeks 12 and 13 were not minor issues, they were bugs that the team considered significant enough to track explicitly, which was not done lightly given the team's practice of resolving smaller problems directly in the code. The absence of tracked defects in Weeks 1 through 11 is consistent with

this practice: smaller issues were absorbed and fixed as part of the normal iteration flow. The late-cycle cluster, by contrast, required dedicated investigation and cross-service fixes, which is precisely the threshold at which the team chose to formalise the work in Jira.

No formal time-tracking was conducted at the individual task level. The planned ten-hours-per-person-per-week figure served as a capacity ceiling rather than a granular estimate, and the team self-managed within that ceiling.

5. Weekly Work Log

The following subsections document the work completed each week, organised by pair. Tables show each pair's members, the tasks they completed or progressed during the week, and the associated Jira ticket identifiers. Start dates and end dates for each ticket are used to assign tickets to weeks. A break was there in Week 7 (March 3–9, 2026) due to reading week.

Week 1 - January 23 – January 26, 2026 - Project Kickoff and Epic Definition

The project commenced on January 23, 2026 with a full-team kickoff session in which the five pairs established the architectural blueprint for the Response to City Disaster platform and defined all twenty Epics that would govern the project lifecycle. Each pair assumed ownership of a portfolio of Epics, providing a clear accountability structure from day one. This initial iteration focused entirely on scoping, decomposition of requirements into trackable work items, and agreement on the technical stack: GCP/GKE infrastructure, Go microservices, React for the web dashboard, Flutter for the mobile application, and Keycloak for identity and access management.

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Pravigya	Epic ownership: Core Evacuation Service, Notification Service, Bug Fixes & QA framework definition.	XP-53, 60, 61
P2	Mansi & Tanmay	Epic ownership: Airflow on GKE (ETL orchestration), Core Incident Registry service definition.	XP-56, 58
P3	Marie & Marta	Epic ownership: GCP Cost Cutting, Services Boilerplate & Deploy, Core Dispatching, Communication Service, Mobile App (Flutter).	XP-47, 52, 59, 62, 64
P4	Niraj & Ajinkya	Epic ownership: Developer Environment Setup, Keycloak Migration to GKE, Dashboard Migration to GKE, Observability Service, Project Documentation.	XP-46, 48, 49, 55, 65
P5	Reetik & Vinol	Epic ownership: GKE VM Scheduling, ClickHouse on GKE, Data ETL Service, IAM Service, Web Dashboard (React).	XP-50, 51, 54, 57, 63

Week 2 - January 27 – February 2, 2026 - Developer Environment Setup & GCP Infrastructure Optimisation

The second week marked the transition from planning to execution. The first work iteration focused on two parallel tracks: establishing consistent, reproducible developer environments across all ten team members, and beginning the GCP infrastructure optimisation initiative by auditing and rationalising the Compute Engine VM footprint. Standardisation of tooling (Git with GPG signing, GitHub Copilot, and Flutter SDK) ensured that all pairs would work from a common baseline for the remainder of the project. The onboarding runbook produced this week became a key reference document for new tooling introduced in subsequent iterations.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Pravigya	Developer environment validation: configured GitHub Copilot in VS Code and JetBrains IDEs; executed and documented the full-stack smoke test against all services.	XP-68, 72
P2	Mansi & Tanmay	Iteration planning and backlog refinement; supported infrastructure audit activities; no direct ticket assignments in this iteration (capacity reserved for upcoming Airflow work).	-
P3	Marie & Marta	Developer tooling: installed and validated Flutter SDK with Android/iOS emulator targets; GCP cost audit: identified idle VMs (< 5% CPU utilisation over 14 days) and decommissioned confirmed unused instances, releasing associated static IPs.	XP-67, 74, 75
P4	Niraj & Ajinkya	Developer tooling: installed Git and configured SSH keys and GPG commit signing across workstations; GCP audit: inventoried all active Compute Engine VMs with usage/owner tags; produced final VM inventory document post-cleanup.	XP-70, 73, 77
P5	Reetik & Vinol	Developer tooling: authored the developer onboarding runbook for workstation setup; GCP cost cutting: right-sized over-provisioned VMs based on the utilisation report generated by the audit.	XP-71, 76

Week 3 - February 3 – February 12, 2026 - Keycloak Migration to GKE & Web Dashboard Migration

With the developer environments consolidated, this week's iteration initiated two major infrastructure migrations simultaneously: moving the Keycloak authentication service from Cloud Run to GKE, and containerising and relocating the Web Dashboard to GKE. The Keycloak migration required particular care to ensure zero data loss; realm configuration was exported prior to the cutover, and the service was subjected to a full load test before the Cloud Run instance was decommissioned. In parallel, pairs worked on the mobile application map integration and early IAM/RBAC design, reflecting the first overlap between infrastructure and feature development streams.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Tanmay	Keycloak GKE migration: assessed current Cloud Run configuration (RAM, CPU, replicas, env vars); configured Kubernetes ingress with TLS termination and DNS cutover; decommissioned Cloud Run Keycloak after successful cutover. Also began mobile app map integration.	XP-78, 82, 84, 90
P2	Ajinkya & Pravigya	Web Dashboard migration: configured environment variables and secrets via Kubernetes Secrets/ConfigMaps for the Dashboard service on GKE. Began Data ETL architecture design.	XP-87
P3	Marta & Mansi	Keycloak migration support: exported Keycloak realm configuration for zero-data-loss migration; conducted full load test on Keycloak on GKE validating all auth flows; wrote K8s Deployment and Service manifests for the Dashboard; executed post-migration Dashboard regression tests.	XP-81, 83, 86, 89
P4	Niraj & Vinol	Web Dashboard migration: containerised the Dashboard application with a multi-stage Dockerfile; set up Kubernetes Ingress with SSL and custom domain routing for the Dashboard service.	XP-85, 88
P5	Reetik & Marie	Keycloak GKE migration: authored Helm chart and K8s manifests for Keycloak; configured persistent storage (CloudSQL/Postgres) for the stateful Keycloak deployment. IAM/RBAC initial design work.	XP-79, 80

Week 4 - February 10 – February 19, 2026 - VM Scheduling Automation, ClickHouse Cluster Deployment & Services Boilerplate

This iteration consolidated the infrastructure layer with three parallel workstreams: the implementation of automated VM start/stop scheduling via GitHub Actions to further reduce GCP costs during off-hours; the deployment and configuration of the ClickHouse analytics database cluster on GKE; and the creation of the Go microservice boilerplate and Kubernetes deployment manifests that would serve as the foundation for all subsequent backend services. The completion of this boilerplate was a critical path item, as it unblocked all five pairs from beginning feature-level service development in subsequent iterations.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Marie	VM scheduling: authored GitHub Actions workflow to stop Compute Engine VMs on schedule; deployed ClickHouse cluster on GKE with persistent volumes; defined the list	XP-92, 97, 102, 107

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
		of remaining services and assigned ownership; coordinated deployment of all services to GKE staging for inter-service communication validation.	
P2	Ajinkya & Tanmay	VM scheduling: designed the VM start/stop schedule aligned with developer hours and timezones; authored GitHub Actions workflow to start VMs at dev start time; added Slack/email notification on VM state change; created Go boilerplate template (REST, health checks, structured logging, tracing); configured per-service CI/CD pipeline (build → test → push → deploy).	XP-91, 95, 103, 106
P3	Marta & Pravigya	VM scheduling: added manual workflow_dispatch trigger for on-demand VM control; ClickHouse: configured automated backups to GCS bucket; documented ClickHouse schema, access credentials, and usage guide; authored multi-stage Dockerfiles for each remaining microservice. Ground Staff feature initial planning.	XP-94, 99, 101, 104
P4	Niraj & Mansi	VM scheduling: authored GitHub Action to start VMs at dev start time; ClickHouse: evaluated deployment options (Operator vs Helm chart) and configured replication and sharding strategy; created Kubernetes manifests (Deployment, Service, HPA, ConfigMap) per service. Extended Dashboard functionality planning.	XP-93, 96, 98, 105
P5	Reetik & Vinol	ClickHouse: connected the cluster to backend services and validated query execution. Support bot and communication service initial architecture. IAM RBAC test case design.	XP-100

Week 5 - February 17 – February 26, 2026 - Bug Fixes, Data ETL Service Implementation & Observability APIs

This iteration marked a pivotal transition: the team shifted from infrastructure setup to active feature development. Two categories of unplanned remediation work emerged - CORS misconfiguration affecting Dashboard-to-backend API calls, and a missing notification bug in the incident creation pipeline. Both issues required investigation before they could be resolved, consuming a portion of the iteration capacity that had been allocated to ETL development. Despite this, the ETL service made strong progress, with the registerDataSource, startIngestion, stopIngestion, and getIngestionStatus APIs all implemented, alongside the TransformRules engine and the first streaming Observability APIs.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Vinol	Data ETL service: identified and documented all required data sources for the ETL pipeline; evaluated shortlisted public APIs for domain data ingestion; implemented the TransformRules engine for schema normalisation; created realistic time-series dummy data for anomaly detection and Observability testing.	XP-116, 122, 126
P2	Ajinkya & Marie	Bug fixes: fixed CORS configuration for Dashboard ↔ Backend API calls; traced and diagnosed the notification pipeline (event → queue → handler → provider); fixed the notification root cause and added structured logging to the notification service; evaluated public APIs for ingestion; implemented startIngestion/stopIngestion control endpoints; verified ETL service triggers data_ingested event. Observability implementation.	XP-110, 113, 114, 117, 120, 123
P3	Marta & Tanmay	Quality assurance: wrote unit and integration tests for the incident notification flow to prevent future regression. Ground Staff Group creation and CRUD operations initial development.	XP-115
P4	Niraj & Pravigya	Bug fixes: reproduced CORS errors across all affected endpoints and documented patterns; added CORS integration tests to the CI pipeline; reproduced the missing notification on incident creation in staging; added dummy/seed data to the Observability service for dashboard testing; implemented streamTrafficFlows WebSocket API for real-time traffic data.	XP-108, 111, 112, 125, 127
P5	Reetik & Mansi	Bug fixes: audited CORS configuration on API Gateway and Istio ingress layer; ETL service: prototyped data ingestion from the top 3 identified data sources; implemented registerDataSource API (store connection config and transform rules); implemented getIngestionStatus API (health check and error counts); designed ETL data schema and ingestion pipeline architecture.	XP-109, 118, 119, 121, 124

Week 6 - February 24 – March 2, 2026 - Observability Streaming APIs, Dashboard & Mobile App Refactoring

Iteration six focused on completing the suite of real-time observability and city-data streaming APIs that would feed both the Web Dashboard and the Mobile application. Ten distinct APIs - covering vehicle locations, power grid data, weather warnings, river levels, hospital statuses, emergency service unit locations, and social media alert correlation - were implemented across all pairs. In parallel, pairs began

the first round of refactoring the Web Dashboard and Mobile application to align their data models with the API outputs delivered in the previous iterations, reducing technical debt accumulated during rapid initial development.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Niraj	Implemented listHospitalStatuses API (capacity and bed counts). Refactored the Mobile application to align with the Dashboard data model and updated state management accordingly.	XP-134
P2	Ajinkya & Vinol	Implemented streamVehicleLocations API (real-time public transport); streamPowerUsage and listPowerOutages (grid data); getCurrentWeather and listActiveWeatherWarnings; streamRiverLevels and listFloodRiskAreas; streamAmbulanceLocations and listAmbulanceLocations. Refactored the Web Dashboard to reflect updated API contracts.	XP-129, 131, 132, 133, 135
P3	Marta & Marie	Implemented Ground Staff members CRUD APIs. Ongoing validation of Ground Staff data flows across backend and frontend surfaces.	-
P4	Mansi & Tanmay	Implemented listTrafficIncidents API (filtered by bounding box) and streamSocialAlerts with correlateAlert for social media signal processing. Extended map functionality in the Mobile application to incorporate new streaming API data.	XP-128, 137
P5	Reetik & Pravigya	Implemented streamCameraImages and getCameraImage APIs (CCTV feeds); streamServiceUnitLocations for Police/Fire units in real time. Designed and began implementation of Dispatch Event Bus architecture.	XP-130, 136

Week 7 - March 3 – March 9, 2026 - Mid-Project Break (Reading Week)

The team observed a scheduled mid-semester break in Week 7. No active iteration was run, and the Jira board was not updated. This break provided an opportunity for informal review of progress to date and lightweight planning for the second half of the project. The infrastructure layer was complete, the observability APIs were largely done, and the team was well-positioned to shift focus fully to core service development upon return. No Jira tickets were active during this period.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
-	All pairs	Scheduled mid-project break. No iteration active.	-

Week 8 - March 10 – March 16, 2026 - Airflow on GKE, IAM Service APIs & Communication Service

Returning from the break, the team deployed Apache Airflow on GKE with a KubernetesExecutor and validated the first end-to-end ETL pipeline DAG, connecting Airflow to ClickHouse, ETL APIs, and GCS storage. Concurrently, the IAM Service - covering OAuth login flows, user profile management, ERT member registration, session management, and account lifecycle - was substantially implemented. The Communication Service Event Bus foundation (topic management, batch publishing, subscription handlers, audit logging) was also established, laying the groundwork for the Incident and Dispatch services that would follow. This week also saw progress on fixing the notification issue identified in Week 5.

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Pravigya	Fixed the notification issue (root cause traced to event handler de-registration under load). IAM Service: implemented initiateOAuthLogin and completeOAuthLogin wrapper APIs. Communication Service: implemented logAuditEvent (async write to immutable audit log).	XP-145, 155
P2	Ajinkya & Niraj	Authored and validated the first ETL pipeline DAG in Airflow with full end-to-end execution. Added new APIs for Observability, Evacuation, and Notifications to the Dashboard and Mobile App.	XP-143
P3	Marta & Vinol	Airflow: configured DAG failure alerts via Slack/email; implemented refreshSession and authorizeRequest APIs with JWT validation; implemented subscribe and unsubscribe handler registration for the Communication Service Event Bus. Integrated Ground Staff flow into the Dashboard and Mobile App.	XP-144, 149, 153
P4	Tanmay & Marie	Airflow: deployed Airflow on GKE with KubernetesExecutor; configured DAG storage via GCS bucket sync. IAM Service: implemented createUpdateUserProfile and addERTDetails (handle ERT ID and Pending status) APIs. Communication Service: implemented publishBatch (high-throughput event grouping); implemented queryAuditLog with filters (ServiceID, TimeRange, EventType). Dashboard minor UI fixes.	XP-140, 146, 147, 152, 156
P5	Reetik & Mansi	Verified that the Observability service correctly served data ingested by the ETL Service (integration test). Airflow: selected deployment strategy and configured connections to ClickHouse, ETL APIs, and GCS. Communication Service: implemented	XP-138, 139, 141, 142, 154, 157

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
		schedule API (delay task execution); implemented createIncident with validateIncident logic. Continued Dispatch Event Bus implementation.	

Week 9 - March 17 – March 23, 2026 - Core Incident Registry, Dispatching & Evacuation Service APIs

Week nine delivered the core operational services of the platform: the complete Incident Registry API set (create, update, list, get, close, escalate, timeline, metrics), the Dispatching Service (unit selection, assignment, notification, status tracking, and metrics), and the Evacuation Service (plan generation, update, publish, and route preview). The geospatial algorithm underpinning the closest-unit lookup for dispatching - `getClosestUnitByType` - was a technically complex deliverable requiring spatial indexing and multi-variable optimisation. The integration test validating the full Incident → Route → Dispatch Unit workflow was also completed this week.

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Tanmay	Incident Registry: implemented <code>getIncident</code> and <code>getIncidentStatus</code> detail endpoints; implemented <code>getIncidentTimeline</code> (audit history of incident lifecycle). Dispatching: implemented <code>updateUnitStatus</code> (En Route / On Scene / Available state machine); implemented <code>getActiveDispatches</code> and <code>getAllUnits</code> . Evacuation: implemented <code>updateEvacuationPlan</code> (adjust based on new hazard data).	XP-160, 163, 170, 171, 175
P2	Ajinkya & Pravigyra	Incident Registry: implemented <code>closeIncident</code> with resolution reporting. Evacuation: implemented <code>publishEvacuationPlan</code> (distribute to Map and Notification services).	XP-161, 176
P3	Marta & Niraj	Incident Registry: implemented <code>updateIncident</code> (status and severity changes). Dispatching: implemented <code>notifyUnit</code> (send dispatch instructions to assigned unit); executed full end-to-end integration test for the Incident → Route → Dispatch Unit flow. Evacuation: implemented <code>generateEvacuationPlan</code> multi-variable pathfinding algorithm; implemented <code>getClosestUnitByType</code> geospatial unit search.	XP-158, 169, 173, 174, 166
P4	Mansi & Vinol	Incident Registry: implemented <code>getIncidentMetrics</code> (aggregated stats); implemented <code>publishIncidentEvent</code> to notify the Event Bus on all state changes. Dispatching: implemented <code>createDispatch</code>	XP-164, 165, 167, 168, 177

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
		(unit selection and locking); implemented assignUnitToIncident (commit assignment). Evacuation: implemented previewEvacuationRoute with traffic management constraints.	
P5	Reetik & Marie	Incident Registry: implemented listActiveIncidents and listIncidents with pagination and filters; implemented escalateIncident (trigger alert levels). Dispatching: implemented getDispatchMetrics (response time statistics and KPI aggregation).	XP-159, 162, 172

Week 10 - March 24 – April 2, 2026 - Notification Service, Communication Service APIs & Web/Mobile Frontend

Iteration ten represented the most expansive iteration of the project, spanning two Jira sub-periods. The Notification Service was completed with send, broadcast, template management, acknowledgement, and batch job APIs. The Communication Service was finalised with WebSocket/REST hybrid messaging. The Web Dashboard frontend received its core screens: Login, Protected Routes, Admin Data Source Configuration, Live Incident Map, Incident Reporting Form, Incident Details View, Traffic/Road Closure Management, and Evacuation Plan Editor. The Mobile application received its Authentication screens (Google Sign-In and Guest Mode) and the TomTom SDK map integration. This iteration demonstrated the highest degree of cross-cutting coordination, with backend pairs handing off API contracts to frontend pairs in real time.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Marie	Web: connected the Admin Configuration Form to the registerDataSource API (backend integration); built the Web Traffic - Road Closure Management interface. Mobile: integrated TomTom SDK with user location tracking for the map screen.	XP-187, 191, 197
P2	Ajinkya & Tanmay	Notification Service: implemented createTemplate and manageTemplates (CRUD) API. Communication: implemented sendMessage (WebSocket/REST hybrid). Web: built the Live Incident Map with real-time incident and unit overlay; implemented the Evacuation Plan Approval Workflow (Manager review screen). Mobile: built the Login Screen with Google Sign-In.	XP-180, 183, 188, 193, 195
P3	Marta & Pravigya	Notification Service: implemented sendNotification (single target), acknowledgeNotification (ERT confirmation), submitBatchJob and getBatchJobStatus.	XP-178, 181, 182, 190, 194, 196

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
		Web: built the Incident Details View (Timeline, Status, Resources). Web: built the Support - Flagged Conversations view for human operators. Mobile: implemented Guest Mode (limited access).	
P4	Mansi & Niraj	Web: built the Login Screen with Google OAuth and ERT login; built the Incident Reporting Form (location picker, type, severity); built the Evacuation Plan Editor (draw zones, assign routes, set waypoints).	XP-184, 189, 192
P5	Reetik & Vinol	Notification Service: implemented broadcastNotification (geo-fenced and group-targeted). Web: implemented Protected Routes using JWT claims (Admin/ERT/Operator roles); built the Admin Data Source Configuration Form (Add/Test/Remove connections).	XP-179, 185, 186

Week 11 - March 31 – April 6, 2026 - Mobile Application Completion & Technical Documentation

In this iteration, the mobile application was brought to a functionally complete state with the addition of periodic map polling, offline caching, evacuation route display, and the support chat interface with integrated WebRTC voice channel. A critical integration test - verifying that an evacuation plan created on the Web Dashboard could be approved and then appear correctly on the Mobile client - was also completed, confirming end-to-end data flow integrity across the full application stack. Concurrently, the technical documentation effort was initiated, with GKE architecture diagrams, CI/CD pipeline documentation, and operational runbooks all produced this week.

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Reetik	Mobile: implemented periodic polling (30-second interval) to fetch and render updated incident and unit data on the map; built the Support Chat Interface (bubble UI) for communication with dispatchers; integrated WebRTC Client for the Voice Channel support feature.	XP-198, 202, 203
P2	Ajinkya & Marie	Mobile: implemented offline caching for map data when connectivity is degraded. Documentation: documented all CI/CD pipelines with annotated flow diagrams.	XP-199, 206
P3	Marta & Tanmay	Integration testing: executed full end-to-end scenario - create Evacuation Plan on Web, approve via Manager Approval Workflow, verify plan appears correctly on Mobile. Documentation: documented GKE cluster	XP-204, 205, 207

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
		architecture and namespace strategy; authored runbooks for common operational tasks (deploy, rollback, scale).	
P4	Niraj & Pravigya	Integration: verified that the Mobile Map correctly displays Incidents created via the Web Dashboard, validating the real-time synchronisation pipeline.	XP-200
P5	Vinol & Mansi	Mobile: built the Evacuation Route View screen (receives and renders approved plans pushed from the backend).	XP-201

Week 12 - April 7 – April 10, 2026 - Technical Documentation Completion & Bug Identification / Resolution

Week twelve completed the documentation layer - secrets management, architecture diagrams, and Confluence publication - while simultaneously surfacing a cluster of inter-related bugs in the Incident and Dispatch lifecycle. These bugs (active dispatches persisting after incident closure, member count mismatches, mobile and dashboard map synchronisation failures, and the missing affiliation field in the mobile app) were all identified through end-to-end manual testing conducted in parallel with documentation work. The convergence of documentation tasks and late-cycle bug discovery placed pressure on the iteration, and the team prioritised the most critical data-integrity bugs for immediate resolution.

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Tanmay & Sriram	End-to-end testing support and bug triage. Assisted with validation of incident and dispatch lifecycle fixes.	XP-214
P2	Ajinkya & Vinol	Bug fixes: added the affiliation field to the Mobile App user profile and registration flow; resolved the mobile app map and Web Dashboard map synchronisation discrepancy (state divergence due to differing polling intervals).	XP-213
P3	Marta & Marie	Documentation: created the overall system architecture diagram covering all services, databases, infrastructure components, and network topology.	XP-209
P4	Mansi & Niraj	Documentation: documented the secrets management approach (Kubernetes Secrets and Secret Manager integration); published all project documentation to the internal Confluence wiki. Bug fixes: resolved the issue of active dispatches not being closed upon incident closure (two manifestations -	XP-208, 210, 212, 215

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
		incorrect state transition and stale UI rendering).	
P5	Reetik & Pravigyaa	Bug fixes: corrected dispatch completion logic to segregate completions by station; fixed the incident closure handler to propagate closure to all active dispatches; resolved the count-of-available-members mismatch (incorrect join between ERT registration and Ground Staff data sets).	XP-211, 216, 217

Week 13 - April 14 – April 17, 2026 - Final Bug Fixes, System Testing & Project Closure

The final iteration of the project focused on resolving the last set of outstanding defects, conducting a formal load test, executing the full test simulation plan, and completing the end-to-end validation of the integrated system. The remaining bugs - duplicate notification delivery, an ERT member approval flow requiring manual intervention unnecessarily, and a UI navigation issue following incident creation - were all resolved. Marta and Vinol executed the load testing scenario, validating system behaviour under concurrent user load. Mansi and Marie executed the Test Simulation Plan, confirming that the system performed correctly against all defined scenarios. The project was delivered in a production-ready state on April 17, 2026.

Pai r	Members	Focus Area / Tasks Completed	Jira Tickets
P1	Sriram & Pravigyaa	Bug fixes: ERT member incident submissions now correctly bypass the manual approval gate (pre-approved status); resolved the incident visibility issue (incidents not appearing without a page refresh - root cause: WebSocket event not triggering a React state update); fixed the 'View Incident' screen rendering error.	XP-218, 220, 225
P2	Ajinkya & Niraj	Bug fix: resolved the duplicate notification delivery issue (notifications being emitted 5 times) caused by multiple event handler registrations on the same topic in the Communication Service.	XP-223
P3	Marta & Vinol	System testing: executed the Load Testing scenario (concurrent user simulation) to validate system throughput, latency, and resilience under stress conditions.	XP-221
P4	Mansi & Marie	System testing: executed the Test Simulation Plan covering all critical user journeys; confirmed full end-to-end system correctness across Web Dashboard and Mobile App.	XP-222, 226
P5	Reetik & Tanmay	Bug fix: corrected the navigation flow so that logging a new incident redirects the user to the incident detail page. Data seeding: populated Ground Staff member data for four	XP-219, 224

Pair	Members	Focus Area / Tasks Completed	Jira Tickets
		facilities around Trinity College Dublin to support the simulation plan execution.	

6. Impact of and Response to Inaccurate Estimates

Since no task-level time tracking was maintained in Jira, the assessment of estimation accuracy relies on qualitative evidence from the project record. In practice, inaccuracies fell into two broad categories: unplanned remediation work, and underestimated complexity in technically novel areas.

On the remediation side, bugs and misconfigurations that were absent from iteration plans occasionally consumed capacity intended for feature development. The team's consistent response was to redistribute tasks within pairs mid-iteration, defer lower-priority items, and add regression tests to the CI pipeline to prevent recurrence - a pattern that contained the impact without requiring scope reduction or deadline extension.

The more structurally significant inaccuracies arose where the team was working with unfamiliar problem domains: geospatial algorithms, multi-objective pathfinding, and the secondary state-machine effects of a distributed incident lifecycle. These areas proved harder to scope accurately upfront, and their consequences - reduced time for edge-case testing, and a cluster of logically related defects surfacing late in the project - reflected the limits of estimation in the absence of prior experience with the same problem class.

The pair rotation model served as an effective structural buffer throughout: because all team members had cross-domain familiarity, unexpected redistributions could be absorbed without prolonged handover delays. The overarching lesson was the value of adversarial integration testing earlier in the lifecycle, before inter-service assumptions have had time to calcify into hard-to-diagnose defects.

7. Conclusion

The Response to City Disaster project was delivered as a functionally complete, tested, and documented cloud-native emergency management platform on April 10, 2026. Over thirteen weeks, the team of ten executed 183 Jira tickets across twenty Epics, deploying eight backend microservices, a React web dashboard, and a Flutter mobile application on GKE. The infrastructure - ClickHouse, Airflow, Keycloak, and a comprehensive CI/CD pipeline - was established in the first six weeks, providing a stable foundation for the feature development that followed.

The rotating pair programming model was a defining characteristic of the team's working practice and contributed meaningfully to code quality, knowledge distribution, and resilience to individual capacity constraints. The planned allocation of ten hours per person per week proved accurate for the majority of iterations, with overages confined to Weeks 9 and 10 where algorithmic complexity and frontend convergence respectively exceeded expectations.

The project diary documented here reflects an honest account of what was planned, what was delivered, and where estimates diverged from reality. The patterns of inaccuracy - unplanned remediation work, algorithmic complexity underestimation, and late-cycle integration defects - are characteristic of greenfield microservices development.